

DSA — Prefix Sum & Difference Array

1D Array • 2D Array • Difference Array • Java Only

Format

1. Pattern
2. Java Code
3. Simple Explanation

Chapter Flow

1. Basic 1D Prefix Sum
2. Prefix Sum for Range Queries
3. Prefix Sum + Index / Pivot
4. Prefix Sum + HashMap — Sum = K
5. Prefix Sum + HashMap — Longest Subarray
6. Prefix Sum + HashMap — Count Subarrays
7. Prefix Sum — 0/1 Transformation
8. Prefix Sum + Modulo K
9. Prefix Sum + Binary Search
10. Prefix Sum + Monotonic Deque
11. Prefix XOR
12. 2D Prefix Sum
13. 2D Rectangle Sum Query
14. 2D Prefix Sum + Row Compression
15. 2D Prefix Sum — Matrix Block / Neighborhood
16. 1D Difference Array — Range Add
17. Difference Array — Range Update + Final Array
18. Difference Array — Interval / Sweep Line
19. 2D Difference Array — Rectangle Update
20. 2D Difference Array + Prefix Reconstruction
21. Prefix + Difference Combined Pattern
22. Competitive Programming Decision Tree
23. Sequence-wise Practice — 35 Problems

1. BASIC 1D PREFIX SUM

Pattern

Build $\text{prefix}[i]$ = sum of elements from 0 to i.

Java Code

```
int n = arr.length;
int[] prefix = new int[n];

prefix[0] = arr[0];

for (int i = 1; i < n; i++) {
    prefix[i] = prefix[i - 1] + arr[i];
}
```

Example / Core Code

```
arr    = [2, 4, 1, 5]

prefix = [2, 6, 7, 12]

prefix[i]
= arr[0] + ... + arr[i]
```

Simple Explanation:

Store the running sum once. Later, use the prefix array instead of adding the same elements again.

2. PREFIX SUM FOR RANGE QUERIES

Pattern

Find sum of [L..R] in O(1) after O(n) preprocessing.

Java Code

```
// Prefix with padding

int[] prefix = new int[arr.length + 1];

for (int i = 0; i < arr.length; i++) {
    prefix[i + 1] = prefix[i] + arr[i];
}

int sum = prefix[R + 1] - prefix[L];
```

Example / Core Code

```
arr = [2, 4, 1, 5, 3]

sum(1, 3)
= 4 + 1 + 5

= prefix[4] - prefix[1]
```

Simple Explanation:

Prefix up to R+1 minus prefix before L gives exactly the range sum.

3. PREFIX SUM + INDEX / PIVOT

Pattern

Compare left sum and right sum using total/prefix information.

Java Code

```
int total = 0;

for (int x : nums)
    total += x;

int leftSum = 0;

for (int i = 0; i < nums.length; i++) {

    int rightSum = total - leftSum - nums[i];

    if (leftSum == rightSum)
        return i;

    leftSum += nums[i];
}

return -1;
```

Example / Core Code

```
// At index i

left  = nums[0 ... i-1]
right = nums[i+1 ... n-1]

leftSum == rightSum
    ↓
pivot / middle index
```

Simple Explanation:

Keep the left prefix sum. The right sum is $\text{total} - \text{left} - \text{current}$.

4. PREFIX SUM + HASHMAP — SUBARRAY SUM = K

Pattern

Count subarrays whose sum equals K. This works with negative numbers too.

Java Code

```
Map<Integer, Integer> freq = new HashMap<>();

freq.put(0, 1);

int prefix = 0;
int ans = 0;

for (int x : nums) {

    prefix += x;

    int need = prefix - k;

    ans += freq.containsKey(need) ? freq.get(need) : 0;

    freq.put(prefix, freq.containsKey(prefix) ? freq.get(prefix) + 1 : 1);
}

return ans;
```

Example / Core Code

```
prefix[j] - prefix[i] = k

therefore:

prefix[i] = prefix[j] - k

Look for:
prefix - k

in the HashMap.
```

Simple Explanation:

If two prefix sums differ by K, the elements between them sum to K. The map stores how many times each prefix sum appeared.

5. PREFIX SUM + HASHMAP — LONGEST SUBARRAY SUM = K

Pattern

Store the earliest index of each prefix sum.

Java Code

```
Map<Integer, Integer> first = new HashMap<>();

first.put(0, -1);

int prefix = 0;
int ans = 0;

for (int i = 0; i < nums.length; i++) {

    prefix += nums[i];

    if (first.containsKey(prefix - k)) {
        ans = Math.max(
            ans,
            i - first.get(prefix - k)
        );
    }

    // Keep FIRST occurrence only
    first.putIfAbsent(prefix, i);
}

return ans;
```

Example / Core Code

```
prefix[i] - prefix[j] = k

For maximum length:
keep the smallest / earliest j.

length = i - j
```

Simple Explanation:

To maximize length, keep the first time a prefix sum appears. Do not overwrite it.

6. PREFIX SUM + HASHMAP — COUNT SUBARRAYS

Pattern

Same prefix identity, but count frequencies instead of storing only an index.

Java Code

```
Map<Integer, Integer> map = new HashMap<>();
map.put(0, 1);

int prefix = 0;
int count = 0;

for (int x : nums) {

    prefix += x;

    count += map.getDefault(
        prefix - k, 0
    );

    map.put(
        prefix,
        map.getDefault(prefix, 0) + 1
    );
}

return count;
```

Example / Core Code

For current prefix P:

need = P - K

Every previous prefix == need
creates one valid subarray.

Simple Explanation:

Difference: longest problems store an index; counting problems store frequency.

7. PREFIX SUM — 0/1 TRANSFORMATION

Pattern

Turn a condition on zeros/ones into a sum problem.

Java Code

```
// Equal number of 0 and 1

// Convert:
0 → -1
1 → +1

int prefix = 0;
Map<Integer, Integer> first = new HashMap<>();

first.put(0, -1);

int ans = 0;

for (int i = 0; i < nums.length; i++) {

    prefix += (nums[i] == 0 ? -1 : 1);

    if (first.containsKey(prefix)) {
        ans = Math.max(
            ans,
            i - first.get(prefix)
        );
    } else {
        first.put(prefix, i);
    }
}

return ans;
```

Example / Core Code

Equal 0s and 1s

0 + 1
→ -1 + 1
→ 0

Therefore:

prefix[j] == prefix[i]
↓
sum of subarray = 0
↓
equal 0s and 1s

Simple Explanation:

Convert the problem into a zero-sum problem. Equal prefix values mean the transformed subarray sums to zero.

8. PREFIX SUM + MODULO K

Pattern

Count subarrays whose sum is divisible by K.

Java Code

```
Map<Integer, Integer> freq = new HashMap<>();

freq.put(0, 1);

int prefix = 0;
int ans = 0;

for (int x : nums) {

    prefix += x;

    int rem = prefix % k;

    if (rem < 0)
        rem += k;

    ans += freq.getOrDefault(rem, 0);

    freq.put(rem,
              freq.getOrDefault(rem, 0) + 1);
}

return ans;
```

Example / Core Code

```
sum(L..R) % K == 0

(prefix[R] - prefix[L-1]) % K == 0

therefore:

prefix[R] % K
==
prefix[L-1] % K
```

Simple Explanation:

Equal remainders mean their difference is divisible by K. For negative values, normalize the remainder.

9. PREFIX SUM + BINARY SEARCH

Pattern

Use prefix sums + binary search when the prefix array is monotonic, usually with positive/non-negative values.

Java Code

```
// Example: minimum length
// with sum >= target

int[] prefix = new int[nums.length + 1];

for (int i = 0; i < nums.length; i++)
    prefix[i + 1] = prefix[i] + nums[i];

int ans = Integer.MAX_VALUE;

for (int i = 0; i < nums.length; i++) {

    int target = prefix[i] + k;

    int j = lowerBound(prefix, target);

    if (j <= nums.length)
        ans = Math.min(ans, j - i);
}
```

Example / Core Code

Positive numbers:

prefix:

0 → 2 → 5 → 6 → 10 → ...

monotonic ↑

Need:

prefix[j] >= prefix[i] + target

→ binary search j

Simple Explanation:

Positive values make prefix sums non-decreasing, so binary search can find the first prefix meeting a target.

10. PREFIX SUM + MONOTONIC DEQUE

Pattern

Shortest subarray with sum $\geq K$ when negative numbers are allowed.

Java Code

```
long[] prefix = new long[n + 1];

for (int i = 0; i < n; i++)
    prefix[i + 1] = prefix[i] + nums[i];

Deque<Integer> dq = new ArrayDeque<>();
int ans = n + 1;

for (int i = 0; i <= n; i++) {

    while (!dq.isEmpty()
        && prefix[i] - prefix[dq.peekFirst()] >= k) {

        ans = Math.min(
            ans,
            i - dq.pollFirst()
        );
    }

    while (!dq.isEmpty()
        && prefix[i] <= prefix[dq.peekLast()]) {

        dq.pollLast();
    }

    dq.offerLast(i);
}

return ans == n + 1 ? -1 : ans;
```

Example / Core Code

```
Why normal sliding window fails:

negative numbers
↓
sum can decrease when window expands

Prefix sums + deque
↓
maintain useful prefix indices
↓
find shortest valid difference
```

Simple Explanation:

Negative numbers destroy the monotonic behavior needed by the classic sum window. The monotonic deque keeps only useful prefix sums.

11. PREFIX XOR

Pattern

Prefix technique also works with XOR because XOR is reversible: $A \oplus B \oplus B = A$.

Java Code

```
int[] px = new int[nums.length + 1];

for (int i = 0; i < nums.length; i++) {
    px[i + 1] = px[i] ^ nums[i];
}

// XOR from L to R
int xor = px[R + 1] ^ px[L];
```

Example / Core Code

```
prefixXor[R + 1]
^
prefixXor[L]

= nums[L] ^ ... ^ nums[R]
```

Simple Explanation:

Use prefix XOR exactly like prefix sum, but subtraction becomes XOR.

12. 2D PREFIX SUM

Pattern

Build a prefix matrix so rectangle sums can be answered quickly.

Java Code

```
int m = matrix.length;
int n = matrix[0].length;

int[][] pre = new int[m + 1][n + 1];

for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {

        pre[i][j] =
            matrix[i - 1][j - 1]
            + pre[i - 1][j]
            + pre[i][j - 1]
            - pre[i - 1][j - 1];
    }
}
```

Example / Core Code

```
pre[i][j]
= top
+ left
- overlap
+ current
```

Simple Explanation:

Use a padded matrix. The overlap is subtracted once because it was added twice.

13. 2D RECTANGLE SUM QUERY

Pattern

Get the sum of any rectangle in $O(1)$ after $O(mn)$ preprocessing.

Java Code

```
// Query rectangle:
// rows r1..r2
// cols c1..c2

int sum =
    pre[r2 + 1][c2 + 1]
    - pre[r1][c2 + 1]
    - pre[r2 + 1][c1]
    + pre[r1][c1];
```

Example / Core Code

```
      c1      c2
r1    +-----+
      |   A   |
      | rectang|
r2    +-----+
```

```
sum = total
      - top
      - left
      + overlap
```

Simple Explanation:

Apply inclusion-exclusion: total rectangle minus the top and left parts, then add the overlap back.

14. 2D PREFIX SUM + ROW COMPRESSION

Pattern

Convert a 2D submatrix problem into repeated 1D subarray problems.

Java Code

```
// Fix top and bottom rows

for (int top = 0; top < m; top++) {

    int[] col = new int[n];

    for (int bottom = top; bottom < m; bottom++) {

        for (int c = 0; c < n; c++)
            col[c] += matrix[bottom][c];

        // Now col[] is 1D.
        // Apply 1D prefix + HashMap here.
    }
}
```

Example / Core Code

```
2D matrix
↓
fix top
↓
fix bottom
↓
compress rows
↓
1D array col[]
↓
solve 1D subarray problem
```

Simple Explanation:

Fix two rows and compress everything between them into one array. Then reuse a 1D prefix-sum technique.

15. 2D PREFIX SUM — MATRIX BLOCK / NEIGHBORHOOD

Pattern

Each cell asks for the sum inside a surrounding rectangle.

Java Code

```
// For each cell (r,c)

int r1 = Math.max(0, r - k);
int c1 = Math.max(0, c - k);

int r2 = Math.min(m - 1, r + k);
int c2 = Math.min(n - 1, c + k);

int sum =
    pre[r2 + 1][c2 + 1]
    - pre[r1][c2 + 1]
    - pre[r2 + 1][c1]
    + pre[r1][c1];
```

Example / Core Code

```
cell
↓
clamp rectangle
↓
4 prefix lookups
↓
sum in O(1)
```

Simple Explanation:

Clamp the rectangle to matrix boundaries, then use the normal 2D rectangle formula.

16. 1D DIFFERENCE ARRAY — RANGE ADD

Pattern

Apply add X to every element in [L, R] using O(1) work per update.

Java Code

```
int[] diff = new int[n + 1];

diff[L] += x;
diff[R + 1] -= x;

// After all updates:
for (int i = 1; i < n; i++) {
    diff[i] += diff[i - 1];
}
```

Example / Core Code

Add X to [L..R]:

```
diff[L] += X; // start
diff[R + 1] -= X; // stop
```

Prefix sum(diff)
→ final array

Simple Explanation:

Do not update every element. Mark where the effect starts and stops, then take one prefix pass.

17. DIFFERENCE ARRAY — RANGE UPDATE + FINAL ARRAY

Pattern

Best when there are many range updates and the final array is needed.

Java Code

```
int[] diff = new int[n + 1];

for (int[] q : queries) {

    int L = q[0];
    int R = q[1];
    int val = q[2];

    diff[L] += val;
    diff[R + 1] -= val;
}

int running = 0;

for (int i = 0; i < n; i++) {

    running += diff[i];

    arr[i] += running;
}
```

Example / Core Code

Q updates × range length
→ $O(QN)$

Difference array:
Q updates → $O(Q)$
rebuild → $O(N)$

Total → $O(Q + N)$

Simple Explanation:

Each update becomes two writes. The final prefix pass applies all updates.

18. DIFFERENCE ARRAY — INTERVAL / SWEEP LINE

Pattern

Count how many intervals are active at each point.

Java Code

```
int[] diff = new int[maxTime + 2];

for (int[] interval : intervals) {

    int start = interval[0];
    int end = interval[1];

    diff[start]++;
    diff[end + 1]--;
}

int active = 0;
int maxActive = 0;

for (int i = 0; i <= maxTime; i++) {

    active += diff[i];

    maxActive = Math.max(
        maxActive,
        active
    );
}
```

Example / Core Code

```
start → +1
end+1 → -1

prefix(diff)
→ active intervals
```

Simple Explanation:

Difference array becomes a sweep line: +1 when an interval starts, -1 after it ends.

19. 2D DIFFERENCE ARRAY — RECTANGLE UPDATE

Pattern

Add X to every cell inside a rectangle in $O(1)$ per update.

Java Code

```
// Rectangle:
// rows r1..r2
// cols c1..c2

diff[r1][c1] += x;

diff[r2 + 1][c1] -= x;
diff[r1][c2 + 1] -= x;

diff[r2 + 1][c2 + 1] += x;
```

Example / Core Code

```
      c1      c2
    +-----+
r1   |      +X      |
    |      |      |
r2   +-----+
```

4 corners:
(+X) (-X)
(-X) (+X)

Simple Explanation:

Mark the four corners. A 2D prefix reconstruction spreads the update across exactly the rectangle.

20. 2D DIFFERENCE + PREFIX RECONSTRUCTION

Pattern

After all rectangle updates, rebuild the matrix using 2D prefix accumulation.

Java Code

```
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {

        if (i > 0)
            diff[i][j] += diff[i - 1][j];

        if (j > 0)
            diff[i][j] += diff[i][j - 1];

        if (i > 0 && j > 0)
            diff[i][j] -= diff[i - 1][j - 1];
    }
}
```

Example / Core Code

```
4-corner updates
↓
2D prefix reconstruction
↓
final matrix
```

Simple Explanation:

Apply every rectangle with four corner changes, then use 2D prefix accumulation to recover the final values.

21. PREFIX + DIFFERENCE COMBINED

Pattern

Use difference for many updates, then prefix to reconstruct/query the result.

Java Code

```
// Step 1: mark updates
diff[L] += x;
diff[R + 1] -= x;

// Step 2: prefix reconstruct
running += diff[i];

// Step 3: use result
arr[i] += running;
```

Example / Core Code

Range update
→ Difference Array

Final values
→ Prefix Sum

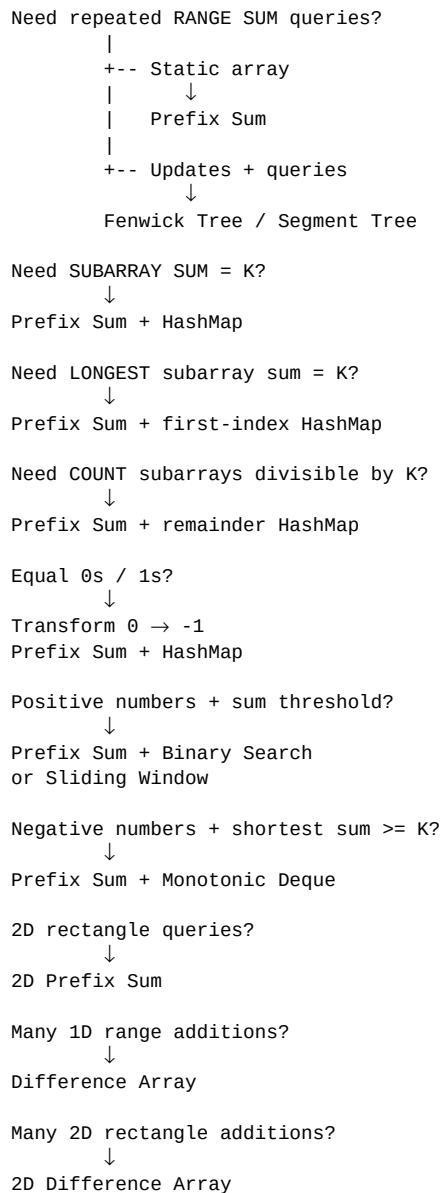
Think:

UPDATE FAST
↓
DIFF
↓
PREFIX
↓
FINAL ARRAY

Simple Explanation:

Difference array is essentially the inverse of prefix accumulation: difference marks changes; prefix rebuilds values.

22. PREFIX SUM DECISION TREE



Core Idea:

Prefix Sum is mainly for fast range/subarray information. Difference Array is mainly for fast range updates.

23. SEQUENCE-WISE PRACTICE — 35 QUESTIONS

Solve in this order: basic → HashMap → modulo → 2D → difference array → advanced.

#	LC	Problem	Pattern
1	1480	Running Sum of 1d Array	Basic prefix
2	303	Range Sum Query - Immutable	1D range query
3	724	Find Pivot Index	Prefix / total
4	1991	Find the Middle Index in Array	Prefix / total
5	1413	Minimum Value to Get Positive Step by Step Sum	Prefix tracking
6	1732	Find the Highest Altitude	Prefix tracking
7	560	Subarray Sum Equals K	Prefix + HashMap
8	325	Maximum Size Subarray Sum Equals k	Prefix + first index
9	525	Contiguous Array	0/1 → -1/+1
10	930	Binary Subarrays With Sum	0/1 + prefix
11	974	Subarray Sums Divisible by K	Prefix + modulo
12	523	Continuous Subarray Sum	Prefix + modulo
13	1590	Make Sum Divisible by P	Prefix + modulo
14	1248	Count Number of Nice Subarrays	Transform + prefix
15	1442	Count Triplets That Can Form Two Arrays of Equal XOR	Prefix XOR
16	1310	XOR Queries of a Subarray	Prefix XOR
17	2433	Find The Original Array of Prefix Xor	Prefix XOR inverse
18	209	Minimum Size Subarray Sum	Prefix / positive values
19	862	Shortest Subarray with Sum at Least K	Prefix + deque
20	303	Range Sum Query - Immutable (re-implement)	Template mastery
21	304	Range Sum Query 2D - Immutable	2D prefix
22	1314	Matrix Block Sum	2D prefix
23	1292	Maximum Side Length of a Square with Sum Less than or Equal to Threshold	2D prefix + binary search
24	1074	Number of Submatrices That Sum to Target	2D → 1D compression
25	363	Max Sum of Rectangle No Larger Than K	2D + ordered structure
26	1109	Corporate Flight Bookings	1D difference
27	1094	Car Pooling	Difference / sweep
28	2381	Shifting Letters II	Difference array
29	370	Range Addition	Difference array
30	598	Range Addition II	2D range concept
31	2132	Stamping the Grid	2D difference / prefix
32	253	Meeting Rooms II	Difference / sweep line
33	731	My Calendar II	Difference / overlap
34	732	My Calendar III	Difference / maximum overlap
35	1589	Maximum Sum Obtained of Any Permutation	Difference + contribution counting

24. ONE-MINUTE REVISION

```
PREFIX SUM
prefix[i] = prefix[i-1] + arr[i]

RANGE SUM
sum(L..R) = prefix[R+1] - prefix[L]

SUM = K
prefix - K → HashMap

LONGEST SUM = K
store FIRST index

COUNT SUM = K
store FREQUENCY

EQUAL 0 / 1
0 → -1
1 → +1

DIVISIBLE BY K
same prefix remainder

PREFIX XOR
rangeXor = px[R+1] ^ px[L]

2D PREFIX
pre[i][j] =
mat[i-1][j-1]
+ pre[i-1][j]
+ pre[i][j-1]
- pre[i-1][j-1]

1D DIFFERENCE
diff[L] += x
diff[R+1] -= x
→ prefix reconstruct

2D DIFFERENCE
4 corners
→ 2D prefix reconstruct

NEGATIVE + shortest sum >= K
→ prefix + monotonic deque
```

Golden Rules

Prefix Sum: calculate cumulative information once → answer range/subarray questions quickly.

HashMap: same prefix value / required prefix → find subarrays.

Difference Array: mark range changes at boundaries → one prefix pass applies all updates.

2D: use the same ideas with rows + columns.

Important

Static range-sum queries are a prefix-sum use case. If the array itself receives many updates and you also need online range queries, prefix sums alone are not enough; Fenwick Tree or Segment Tree is the usual next pattern.